HDOC Day-50

Name	Gurveer Singh Cheema
SAP ID	590032925
Batch	15

Problem Statement

Write a C program that reads the **size** and elements of a variable-length integer array **nums[]**. Pass the array to a function named **majority_element()** using its address. Use pointers to pass the array between the calling function and **majority_element()**.

The function must count the occurrences of each distinct element, find an element that occurs strictly more than **size / 2** times, display that majority element if it exists, and display **-1** otherwise.

C Program

```
#include <stdio.h>

void majority_element(int *nums, int size)
{
    int i, j, count;

    for (i = 0; i < size; i++)
    {
        count = 0;

        for (j = 0; j < size; j++)
        {
            if (*(nums + i) == *(nums + j))
                count++;
        }

        if (count > size / 2)
        {
            printf("Majority element: %d\n", *(nums + i));
            return;
        }
    }

    printf("-1\n");
}

int main()
{
    int size, i;

    printf("Enter size of array: ");
    scanf("%d", &size);

    int nums[size];

    printf("Enter array elements: ");
    for (i = 0; i < size; i++)
        scanf("%d", nums + i);

    majority_element(nums, size);

    return 0;
}
```

Explanation

The array name **nums** acts as the address of its first element. It is passed to **majority_element()** as a pointer. For every element, the nested loop counts how many times it occurs. If its count is strictly greater than **size / 2**, it is printed as the majority element. If no element satisfies this condition, the program prints **-1**.

Sample Output

```
Example 1
Enter size of array: 5
Enter array elements: 3 3 2 3 3
Majority element: 3

Example 2
Enter size of array: 7
Enter array elements: 2 2 1 1 1 2 2
Majority element: 2

Example 3
Enter size of array: 6
Enter array elements: 1 2 3 2 3 3
-1
```

Result

Thus, the C program successfully finds the majority element using pointers and displays **-1** when no majority element exists.